

AI Research Engineer Roadmap

A realistic portfolio, CV, and career plan for remote AI work now and frontier-lab roles later

Personal context

Martín - Electronic Engineering and Mathematics student at Universidad Nacional de Colombia, Sede Manizales, Campus La Nubia. Goal: work remotely in AI while finishing university, then become competitive for research engineering roles at companies such as OpenAI, Anthropic, Google DeepMind, or similar frontier AI teams.

The honest thesis

The goal is realistic as a 2- to 5-year trajectory, not as a quick CV trick. The practical path is: build public proof -> get remote AI/startup/open-source/research-adjacent experience -> apply to fellowships, residencies, or early-career roles -> later target San Francisco or another AI hub.

Use this document as a working roadmap. Update the projects, metrics, links, and application targets every month.

1. Reality check: good goal, sharper strategy

Your mentality is good: you are thinking beyond classes and toward evidence. But frontier AI labs do not hire people because they studied AI. They hire people who can already build, evaluate, debug, and explain AI systems with independence.

Do not optimize for looking interested. Optimize for being visibly useful.

A strong application is not only a CV. It is a package of public technical proof: GitHub repositories, reproducible experiments, technical writing, clean engineering, and a clear specialization.

What is realistic while studying in Colombia

Goal	Realism	Interpretation
Direct remote frontier-lab research role while still studying	Possible, but rare	Requires exceptional public proof, referrals, or a very specific fit.
Remote AI/ML role at a startup or applied AI company	More realistic	Good first paid step while finishing university.
Open-source contribution path	Very realistic	Can create public credibility without needing permission.
Residency/fellowship after portfolio	Realistic target	Especially relevant for people from math, engineering, physics, or adjacent fields.
San Francisco later	Realistic	Better after graduation, strong projects, internships, or remote experience.

Career pages change often. As of source checks made on June 25, 2026, OpenAI advertises early-career opportunities and residency-style pathways, Anthropic lists Fellows programs and notes that it does not currently offer internships, and many Google/DeepMind research listings emphasize strong ML/software experience and often advanced research credentials.

2. Best target role: Research Engineer

The strongest initial positioning is not generic 'AI developer' and not pure 'research scientist' yet. The best target is:

Target identity

Research-oriented AI/ML engineer focused on foundation models, ML systems, and scientific/engineering applications of AI.

Why this fits your background

- Mathematics gives you the foundation for optimization, probability, statistics, information theory, and paper reading.
- Electronic engineering gives you systems intuition: signals, hardware constraints, control, embedded systems, circuits, and performance tradeoffs.
- Together, these make you more differentiated than someone who only fine-tunes chatbots.

Role ladder

Role	Core work	Fit
AI/ML Engineer	Build applied AI systems: RAG, fine-tuning, APIs, deployment, evaluation.	Most realistic remote first role.
Research Engineer	Implement papers, run experiments, build ML systems, improve training/evaluation pipelines.	Best medium-term target.
ML Systems / Inference Engineer	Optimize serving, GPUs, quantization, batching, compilers, kernels, distributed systems.	Great fit with engineering background.
Research Scientist	Originate research agendas, publish papers, design new methods.	Longer-term; often PhD-like evidence.

Your wedge

AI for engineering and scientific reasoning + reliable model evaluation + ML systems. This is more distinctive than 'I build chatbots.'

3. Your evidence package

The CV should summarize evidence. So the first job is to build the evidence. Your public profile should have five components.

Asset	What it must prove
1. One-page CV	Short, technical, role-specific. It should point to proof, not replace it.
2. GitHub	Four to six polished repos with READMEs, experiments, results, and reproducibility instructions.
3. Personal website	A simple page with projects, writing, CV, and contact. No need for fancy animations.
4. Technical writing	Articles explaining what you built, what failed, and what you learned.
5. Open-source activity	Issues, PRs, docs, benchmarks, bug fixes, examples, or evaluation contributions.

Repo quality standard

- Every repo must explain the problem, method, architecture, dataset, metrics, failure cases, reproduction steps, and next steps.
- Use clean structure: `src/`, `configs/`, `tests/`, `notebooks/`, `results/`, `README.md`, `report.md`.
- A portfolio without error analysis looks like a demo. A portfolio with error analysis looks like engineering.

Five-minute test

Could an engineer at a frontier AI lab open your GitHub for five minutes and say: 'This person can probably help us'? That is the bar.

4. Four flagship projects to build first

Project	What to include
1. MiniGPT from scratch	Implement tokenizer, embeddings, causal attention, multi-head attention, residuals, normalization, training loop, generation, and perplexity/evaluation. Purpose: prove transformer fundamentals.
2. Evaluated RAG system	Build ingestion, chunking, embeddings, retrieval, reranking, citations, answer generation, retrieval metrics, faithfulness checks, latency, and failure cases. Use engineering/math/electronics documents.
3. LoRA fine-tuning project	Fine-tune a small open model on a curated engineering/math/electronics dataset. Include baseline, dataset card, model card, LoRA/QLoRA setup, evaluation, and error analysis.
4. Inference benchmark	Deploy a model with FastAPI and Docker. Compare FP32/FP16/quantized inference, batching, CPU/GPU where possible, latency, throughput, and cost tradeoffs.

Optional fifth and sixth projects after the first four

- Paper reproduction: reproduce a small version of LoRA, Attention Is All You Need, DDPM, CLIP-style contrastive learning, or Vision Transformer.
- AI safety/evaluation harness: test hallucination, refusal behavior, prompt injection, bias, calibration, or RAG faithfulness.

Project principle

Learn every concept twice: once from first principles, once as a professional tool. Example: attention from scratch -> PyTorch Transformer -> Hugging Face fine-tuning -> deployed LLM/RAG evaluation.

5. Twelve-month execution plan

Time	Theme	Deliverables
Months 1-2	Foundations and setup	Python for ML, NumPy, PyTorch, autograd, Git/GitHub, Linux basics. Deliver: NumPy neural network, PyTorch MLP/CNN repo, CV v1.
Months 3-4	Core deep learning	CNNs, ResNets, optimization, regularization, experiment tracking. Deliver: CIFAR-10 or technical vision project with strong README.
Months 5-6	Transformers	Tokenization, attention, decoder-only Transformer, language modeling. Deliver: MiniGPT from scratch and technical article.
Months 7-8	LLM applications	RAG, embeddings, vector search, reranking, LoRA/QLoRA, evaluation. Deliver: evaluated RAG and LoRA projects.
Months 9-10	Systems and deployment	FastAPI, Docker, ONNX, quantization, vLLM/Triton basics, MLflow/W&B.; Deliver: inference benchmark.
Months 11-12	Visibility and applications	Portfolio v2, targeted CVs, open-source PRs, mock interviews, remote applications, fellowship/residency tracking.

Weekly rhythm

Share	Activity	Purpose
40%	Implementation	Code the model, pipeline, or system.
25%	Theory	Derive equations and understand the why.
20%	Experiments	Track metrics, debug failures, compare baselines.
10%	Papers/docs	Read official docs and selected papers.
5%	Writing	Update README, notes, blog, or project report.

6. First 30 days: start the machine

Period	Actions
Week 1	Create GitHub, LinkedIn, simple CV draft, learning notes. Study PyTorch tensors, autograd, nn.Module, training loops. Build linear/logistic regression.
Week 2	Build an MLP from scratch in NumPy and the same MLP in PyTorch. Write: 'Backpropagation from first principles'.
Week 3	Build a CNN on MNIST or CIFAR-10. Add training curves, validation metrics, README, and notes on overfitting.
Week 4	Polish one repo. Publish one technical article. Make one LinkedIn post explaining the project and what failed.

Do not wait until you feel ready

The first repo does not need to be perfect. It needs to be clean, honest, reproducible, and improved over time. Public compounding beats private perfection.

Non-negotiable habits

- Use Git from day one: small commits, clear messages, branches for experiments.
- Keep a lab notebook: hypothesis, experiment, result, interpretation, next action.
- Always compare to baselines before claiming a model is good.
- Write down failure cases. They are not embarrassing; they are the source of engineering maturity.
- Study English technical communication: your writing is part of your competence signal.

7. CV, positioning, and application strategy

CV headline

Suggested positioning

Electronic Engineering and Mathematics student at Universidad Nacional de Colombia, focused on deep learning, foundation models, ML systems, and engineering/scientific AI applications.

Three CV versions

CV version	Emphasize
Research Engineer	PyTorch/JAX, transformers, experiments, paper reproduction, evaluation, math, distributed training basics.
AI/ML Engineer	RAG, fine-tuning, deployment, APIs, Docker, monitoring, latency, product reliability.
ML Systems	GPU inference, quantization, batching, ONNX, vLLM/Triton basics, profiling, performance benchmarks.

Application layers

Layer	Target	Reason
Layer 1	Remote AI startups and applied AI companies	Most realistic first paid AI experience.
Layer 2	Open-source AI tooling companies	Hugging Face-like ecosystems, MLOps, inference, evaluation, developer tools.
Layer 3	Research assistantships and labs	University labs, external research groups, collaborations, paper/code reproduction.
Layer 4	Fellowships, residencies, emerging talent	Bridge from strong student/adjacent-field profile to frontier-lab roles.
Layer 5	OpenAI, Anthropic, Google DeepMind	Apply once your public proof is strong enough to make the interview obvious.

8. Skill checklist

Area	Competence target
Math	Linear algebra, probability, statistics, optimization, information theory, numerical methods.
Core ML	Train/validation/test, baselines, metrics, data leakage, calibration, error analysis.
Deep learning	MLPs, CNNs, ResNets, attention, Transformers, tokenization, training loops, regularization.
LLM stack	Hugging Face, RAG, embeddings, vector search, reranking, LoRA/QLoRA, model evaluation.
Systems	Linux, Git, Docker, FastAPI, experiment tracking, ONNX, quantization, vLLM/Triton basics.
Research	Paper reading, reproduction, ablations, baselines, failure analysis, clear technical writing.
Interviews	Algorithms, ML theory, PyTorch implementation, system design, research discussion.

Monthly checkpoint

At the end of each month, ask: What did I build? What did I measure? What did I publish? What did I learn from failure? What proof can a stranger inspect?

The north star
Become the kind of person who can take an ambiguous AI problem, read the relevant papers/docs, implement a baseline, run controlled experiments, diagnose failures, write the result clearly, and ship a usable system.

9. One-page summary

Core path

UNAL student -> public AI portfolio -> remote AI/startup/open-source experience -> fellowship/residency/early-career role -> research engineering or ML systems role -> San Francisco or frontier-lab path.

First four artifacts to create

- MiniGPT from scratch in PyTorch.
- Evaluated RAG system for engineering/math/electronics documents.
- LoRA fine-tuning project on an engineering-domain dataset.
- Dockerized inference benchmark with latency and quantization results.

Key mindset

Do not ask only 'Can I get into OpenAI?'. Ask weekly: 'What would make me 1% more like a research engineer?' Then build that evidence publicly.

Source notes checked on June 25, 2026

Source	Why it matters
OpenAI Careers	Careers page describes the OpenAI Residency as a six-month pathway for researchers and engineers who do not currently focus on AI.
OpenAI Emerging Talent	Early-career page describes full-time roles in research, applied engineering, and product for people with 0-3 years of experience.
Anthropic Careers	Careers FAQ states Anthropic does not currently offer internships; jobs page lists Fellows programs and research/engineering roles.
Google Careers / DeepMind listings	Example research roles emphasize strong degrees or equivalent experience, ML/software experience, frameworks such as PyTorch/JAX/TensorFlow, and sometimes publications or PhD-level evidence.

Always re-check official career pages before applying. Role locations, remote policies, internships, fellowships, and eligibility requirements change frequently.